



Future and Conclusion



● In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

● **This enables us to...**

Develop and verify optimisations for languages using BoC

Develop a memory model for languages that use BoC

Further refine BoC, e.g. through concurrent execution of behaviours that only read data

when Behaviours Have to Happen

An Axiomatic Model of Causality in Behaviour-Oriented Concurrency

Luke Cheeseman 

Uppsala University, Sweden

Elias Castegren 

Uppsala University, Sweden

Tobias Wrigstad 

Uppsala University, Sweden

Sophia Drossopoulou 

Imperial College London, UK

Matthew J. Parkinson 

Azure Research, London, UK

Abstract

Behaviour-oriented concurrency (BoC) is a recently established programming model in which programmers define concurrent operations that execute atomically across multiple isolated resources. This allows for expressive interactions but introduces complex causal dependencies determined by dynamic resource overlap. Previous work defines the causal guarantees of BoC operationally, but mixes intended design constraints with incidental implementation details, leading to unintended causal orders. BoC is now being implemented across multiple languages and runtimes, all relying on the operational descriptions of causality. This paper develops an axiomatic model of BoC executions that makes the intrinsic orders explicit and derives the intended causal relation from their interaction. Using a set of representative programs and candidate executions, we motivate the design of this causal relation. We then prove that a representative minimal core calculus for BoC is sound with respect to this axiomatic model. Together, these results provide an implementation-independent foundation for reasoning about BoC causality across runtimes, schedulers and optimisation decisions.

2012 ACM Subject Classification Theory of computation → Concurrency

Keywords and phrases Concurrency, Parallelism, Language Design, Causality

Digital Object Identifier 10.4230/LIPIcs.CONCUR.2026.23

Supplementary Material *Software (Proofs)*: https://github.com/fxpl/axiomatic_boc [7]

archived at [arXiv:2403.04716](https://arxiv.org/abs/2403.04716)

Acknowledgements We thank David Black-Schaffer and the reviewers for helpful discussions and feedback on the paper.

1 Introduction

Modern concurrent programming must balance the efficiency benefits of parallelism against the complexity of reasoning about program execution. Behaviour-oriented concurrency (BoC) is a programming model in which tasks declare required isolated resources; tasks with disjoint requirements may execute concurrently, while tasks with overlapping requirements are subject to explicit ordering guarantees, simplifying reasoning about program execution [10]. Previous work on BoC defines these ordering guarantees operationally, but mixes intended design constraints with incidental implementation details, inducing unintended causal orders [10]. BoC is now being developed in multiple languages and runtimes, including C++ and Python runtimes, and research languages such as Verona [22, 12, 13, 4]. As these efforts evolve, they rely on the informal and overly restrictive descriptions of causality inherent in the operational



© Luke Cheeseman, Elias Castegren, Tobias Wrigstad, Sophia Drossopoulou, and Matthew J. Parkinson;

licensed under Creative Commons License CC-BY 4.0

37th International Conference on Concurrency Theory (CONCUR 2026).

Editors: Ana Sokolova and Patrick Totzke; Article No. 23; pp. 23:1–23:17



Leibniz International Proceedings in Informatics

LIPIcs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Future and Conclusion

- In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

- This enables us to...

Develop and verify optimisations for languages using BoC

Develop a memory model for languages that use BoC

Further refine BoC, e.g. through concurrent execution of behaviours that only read data



Future and Conclusion

- In this paper, we have...

Constructed an axiomatic model for Behaviour-oriented Concurrency

Captured the intended causal order between behaviours

Proven that our earlier work, and its derivatives, are sound with respect to this model

Provided an implementation-independent foundation for reasoning about BoC programs

- This enables us to...

Develop and verify optimisations for languages using BoC

Develop a memory model for languages that use BoC

Further refine BoC, e.g. through concurrent execution of behaviours that only read data

Thank you for listening

